



María Eugenia Herrera Cifuentes

Carnet N°202605716

Estructura de datos

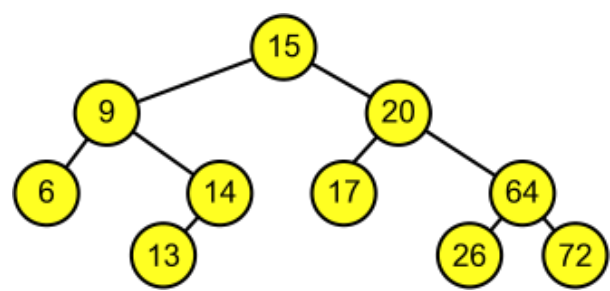
Ficha Técnica

Ing. Brandon Antony Chitay Coutiño

14-05-2026

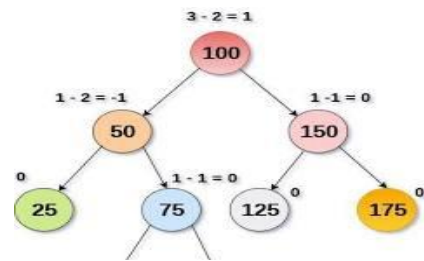
UDV

1. Árbol Binario de Búsqueda (BST)



Componente	Descripción Detallada
Identificación	Estructura jerárquica de nodos donde cada elemento padre tiene como máximo dos hijos. Cumpliendo que el hijo izquierdo es menor y el derecho es mayor al nodo padre Es la base de las estructuras de búsqueda dinámica.
Complejidad (Big O)	Búsqueda: $O(\log n)$ (Promedio) / $O(n)$ (Peor) Inserción: $O(\log n)$ (Promedio) / $O(n)$ (Peor) Eliminación: $O(\log n)$ (Promedio) / $O(n)$ (Peor)
Mecánica	Se organiza mediante una relación de orden: el subárbol izquierdo contiene valores menores al nodo actual y el derecho valores mayores.
Ventaja / Límite	✓ Ventaja: Gran facilidad de implementación y eficiencia en datos desordenados. ✗ Límite: Si los datos entran ordenados, el árbol se "degenera" en una lista, perdiendo su eficiencia.
Uso Industrial	Sistemas de gestión de símbolos en compiladores básicos o diccionarios de datos simples en memoria.

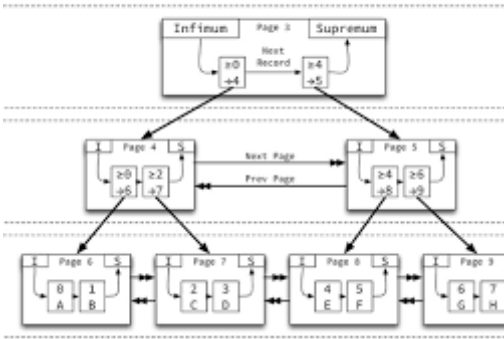
2: Árbol Balanceado (AVL)



Componente	Descripción Detallada
Identificación	Árbol binario de búsqueda auto-equilibrado. Debe su nombre a sus inventores Adelson-Velsky y Landis.
Complejidad (Big O)	Búsqueda: $O(\log n)$ (Peor caso garantizado) Inserción: $O(\log n)$ Eliminación: $O(\log n)$
Mecánica	Mantiene un Factor de Equilibrio : la diferencia de altura entre subárboles no puede exceder 1. Si ocurre, ejecuta rotaciones (LL, RR, LR, RL).
Ventaja / Límite	✓ Ventaja: Garantiza que el árbol nunca se degrade; Las búsquedas siempre son rápidas. ✗ Límite: Las inserciones son más lentas por el costo de reequilibrar tras cada cambio.
Uso Industrial	Bases de datos de alto tráfico de lectura , donde buscar datos es mucho más frecuente que insertarlos.

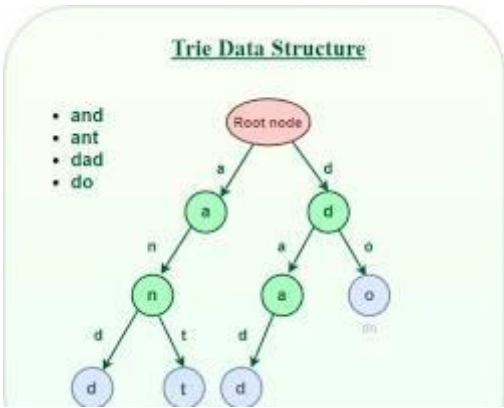
3: Árbol B / B+

B+Tree Structure



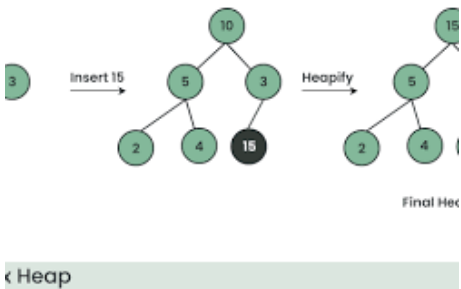
Componente	Descripción Detallada
Identificación	Árboles multicanal balanceados diseñados específicamente para el manejo de grandes volúmenes de datos en almacenamiento secundario (discos).
Complejidad (Big O)	Búsqueda: $O(\log n)$ Inserción: $O(\log n)$ Eliminación: $O(\log n)$
Mecánica	Los nudos poseen un alto factor de ramificación (muchos hijos). En el B+ , los datos reales residen solo en las hojas, que están ligadas entre sí.
Ventaja / Límite	✓ Ventaja: Minimiza los accesos lentos a discoteca; ideal para lectura secuencial masiva. ✗ Límite: Implementación extremadamente compleja y mayor uso de memoria por nodo.
Uso Industrial	Estructura fundamental de Motores de Bases de Datos (PostgreSQL, SQL Server) y sistemas de archivos (NTFS, APFS).

4: Trie (Árbol de Prefijos)



Componente	Descripción Detallada
Identificación	Árbol de búsqueda donde la posición de un nodo define la clave asociada. Se utiliza principalmente para cadenas de texto.
Complejidad (Big O)	Búsqueda: $O(k)$ (donde k es el largo de la palabra) Inserción: $O(k)$ Eliminación: $O(k)$
Mecánica	Cada nodo representa un carácter. Las palabras con prefijos comunes comparten la misma ruta desde la raíz, ahorrando espacio en prefijos repetidos.
Ventaja / Límite	✓ Ventaja: Búsqueda extremadamente rápida que no depende de cuántos datos haya (n), sino del largo de la palabra (k). ✗ Límite: Consume mucha memoria si el alfabeto es grande.
Uso Industrial	Algoritmos de Autocompletado , correctores ortográficos y enrutamiento de direcciones IP.

5. Heap (Montículo Min/Max)



Componente	Descripción Detallada
Identificación	Árbol binario completo especializado en mantener el acceso rápido al elemento de mayor (o menor) prioridad.
Complejidad (Big O)	Acceso Máx/Mín: $O(1)$ Inserción: $O(\log n)$ Eliminación: $O(\log n)$
Mecánica	Se suele implementar en un arreglo por eficiencia. Tras insertar, el elemento "flota" hacia arriba hasta que su padre es mayor (Max-Heap) o menor (Min-Heap).
Ventaja / Límite	✓ Ventaja: Acceso instantáneo al elemento prioritario. ✗ Límite: No permite búsquedas eficientes de elementos que no estén en la raíz ($O(n)$).
Uso Industrial	Colas de Prioridad en Sistemas Operativos para gestionar procesos y el algoritmo de ordenamiento Heapsort .